

# Mini-Project 3: Choose Your Own Adventure — Collections and Reusable Code

Russell Helmstedter

*09/25/2026*

## Contents

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Overview</b>                                            | <b>2</b> |
| <b>2</b> | <b>Describe the Student Assignment</b>                     | <b>3</b> |
| 2.1      | Chatbot 2.0 . . . . .                                      | 3        |
| 2.1.1    | Target Grade Level . . . . .                               | 3        |
| 2.1.2    | Subject or Classroom Context . . . . .                     | 3        |
| 2.1.3    | Platform . . . . .                                         | 3        |
| 2.1.4    | Estimated Completion Time . . . . .                        | 3        |
| 2.1.5    | Learning Objectives . . . . .                              | 3        |
| 2.1.6    | The Problem . . . . .                                      | 3        |
| 2.1.7    | Required Programming Features . . . . .                    | 4        |
| 2.1.8    | Final Product . . . . .                                    | 4        |
| 2.1.9    | Success Criteria . . . . .                                 | 4        |
| 2.1.10   | Written Reflection . . . . .                               | 5        |
| 2.1.11   | Optional Extension . . . . .                               | 5        |
| <b>3</b> | <b>Identify the Assessed Concepts</b>                      | <b>5</b> |
| 3.1      | From Chapter 9: Collections . . . . .                      | 5        |
| 3.2      | From Chapter 10: Functions and Decomposition . . . . .     | 6        |
| 3.3      | Retained from earlier chapters . . . . .                   | 6        |
| <b>4</b> | <b>Provide a Detailed Specification</b>                    | <b>6</b> |
| 4.1      | Program Behavior . . . . .                                 | 6        |
| 4.2      | Input . . . . .                                            | 6        |
| 4.3      | Processing . . . . .                                       | 7        |
| 4.4      | Output . . . . .                                           | 7        |
| 4.5      | Collection Specification . . . . .                         | 7        |
| 4.6      | Function Specification . . . . .                           | 8        |
| 4.7      | Algorithm . . . . .                                        | 8        |
| <b>5</b> | <b>Include Example Input and Output</b>                    | <b>8</b> |
| 5.1      | Case 1 — Typical . . . . .                                 | 9        |
| 5.2      | Case 2 — Special: messy typing . . . . .                   | 10       |
| 5.3      | Case 3 — Boundary: the answer is not on the menu . . . . . | 10       |
| 5.4      | Case 4 — Error: the user answers nothing . . . . .         | 11       |

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| <b>6</b> | <b>Create the Solution</b>                                                            | <b>11</b> |
| 6.1      | Source Code . . . . .                                                                 | 11        |
| 6.2      | Evidence of Testing . . . . .                                                         | 13        |
| <b>7</b> | <b>Create Teacher Notes</b>                                                           | <b>14</b> |
| 7.1      | Required Prior Knowledge . . . . .                                                    | 14        |
| 7.2      | New Vocabulary . . . . .                                                              | 14        |
| 7.3      | Suggested Scaffolding . . . . .                                                       | 14        |
| 7.4      | A Likely Loop Misconception . . . . .                                                 | 14        |
| 7.5      | A Likely Function Misconception . . . . .                                             | 14        |
| 7.6      | A Note on the <code>else</code> Branches . . . . .                                    | 15        |
| 7.7      | Accessibility . . . . .                                                               | 15        |
| 7.8      | Support for Students Who Need More Guidance . . . . .                                 | 15        |
| 7.9      | Extension for Students Ready for More . . . . .                                       | 15        |
| <b>8</b> | <b>Reflection</b>                                                                     | <b>16</b> |
| 8.1      | Why is the assignment appropriate for this grade level? . . . . .                     | 16        |
| 8.2      | How does the platform support or limit the Chapter 9 and 10 concepts? . . . . .       | 16        |
| 8.3      | How does the assignment help students organize data or decompose a problem? . . . . . | 16        |
| 8.4      | What errors did I discover while testing? . . . . .                                   | 17        |
| 8.5      | What would I revise before using it? . . . . .                                        | 17        |
| <b>9</b> | <b>Files</b>                                                                          | <b>18</b> |

## 1 Overview

|            |                                                                    |
|------------|--------------------------------------------------------------------|
| Path       | D — High School, grades 9-12                                       |
| Platform   | Python on Google Colaboratory                                      |
| Assignment | Chatbot 2.0                                                        |
| Concepts   | Lists, iteration, membership, functions, parameters, return values |

This is a real assignment for my AP CSP sections, not a hypothetical one. Students built a chatbot earlier this quarter using variables, string methods, f-strings, and `if~/~elif~/~else`. Since then we have covered lists, loops, functions, and the `random` and `time` modules. Chatbot 2.0 sends them back into code they already wrote, with tools that make it better.

The pedagogical bet of this project is iteration. Software is not written once, and the most useful thing a first-year programmer can learn about their own work is that version 1 was limited by what they knew, not by what they wanted. Students have a concrete artifact of that limit sitting in their Drive: eight nearly identical stretches of ask-and-clean code, written three weeks ago, by them. Version 2 does not introduce functions as a new topic to admire. It hands students repetition they created themselves and gives them the tools to remove it.

## 2 Describe the Student Assignment

### 2.1 Chatbot 2.0

#### 2.1.1 Target Grade Level

9-12. AP Computer Science Principles, first semester.

#### 2.1.2 Subject or Classroom Context

Follows the Chatbot assignment from earlier in the quarter, plus the notes notebooks on lists, loops, functions, and modules. Students may extend their own version 1 or start fresh.

#### 2.1.3 Platform

Python 3 in Google Colaboratory, on Chromebooks.

#### 2.1.4 Estimated Completion Time

Three 55-minute periods. Version 1 ran long, and this asks for more. One period to get the asking function working across the whole bot, one to build the reply lists and the menus, one to finish the retry loops and write the reflection.

#### 2.1.5 Learning Objectives

You will be able to:

- write a function that takes information in, does a job with it, and hands a result back;
- store related values in a list and go through that list with a loop;
- check whether what somebody typed is in a list, and act on the answer;
- explain why your version 2 is better than your version 1, in terms a user would notice.

#### 2.1.6 The Problem

Software does not get written once. The apps on your phone update every couple of weeks, and each of those updates is a team deciding their last version could be improved upon.

Your chatbot was version 1. It worked. You had variables, `if` statements, and string methods, and you built something that could hold a conversation out of them.

That is iterative thinking. You build with what you know, you learn more, and you come back. Version 1 was not limited by what you wanted it to do. It was limited by what you knew how to do.

You know more now: lists, dictionaries, loops, functions, and modules like `random` and `time`. So build version 2, improving both the design and the user experience. From a design standpoint, cut repeated code by using loops and writing functions. From a user experience standpoint, make it feel less like filling out a form and more like talking to something that is paying attention.

Start from your version 1 or start from scratch. Keep the same bot or invent a new one. Your call — what gets graded is the version you turn in.

### 2.1.7 Required Programming Features

Your version 2 has to meet every requirement from version 1 — greet the user and get their name first, at least 8 questions asked with `input()`, every answer cleaned with string methods before it is compared, at least four decision blocks with at least two using `elif` and every one ending in `else`, f-strings wherever you refer to something the person typed, and code that runs without errors. None of that earns points this time. It is the floor. A bot that does not clear it comes back to you ungraded.

On top of that:

1. **Write a function that asks a question and returns the cleaned answer.** It takes the question as a parameter. Use it for every question your bot asks.
2. **Give your bot at least two lists of possible replies, with at least three replies in each,** and use `random.choice()` to pick from them. Use them for lines that work no matter what the person said — lines that depend on the answer belong in your decision blocks.
3. **Show the person their options on at least two of your questions.** Keep those options in a list and loop over it to print them.
4. **Give the person more than one try on those same questions.** If what they typed is not on the list, say so and ask again instead of moving on.

### 2.1.8 Final Product

One Colab notebook containing the program and a written reflection, submitted through Google Assignments in Canvas.

### 2.1.9 Success Criteria

Your chatbot is finished when all of the following are true.

- ☐ A person who has never used it can talk to it start to finish without getting stuck.
- ☐ Your asking function is written once and used for every question, including the name.
- ☐ Adding a ninth question would take you one line.

- ☐ Running the bot twice with the same answers does not produce identical output.
- ☐ Typing something that is not on the menu gets you asked again, not ignored.
- ☐ Nothing the person types can send the bot into a loop it cannot leave.
- ☐ The program runs top to bottom without errors after **Runtime -> Restart and run all**.

### 2.1.10 Written Reflection

1. Technical & Code Logic
  - Version 1 asked and cleaned each answer separately. Compare that to the function you wrote this time. What changed, and what would change again if you added two more questions?
  - You used lists in more than one way this time: some you pick from at random, one you print and then check an answer against. What is each list holding, and why is a list the right place to keep it?
2. Design & User Experience
  - What did you put in your lists of replies, and why those? How does it change what the bot feels like to talk to?
  - You picked two questions worth a second try. Pick one of them. Why that question? What would it cost the person if you did it to every question?
3. Iteration & Future Versions
  - What is one thing you would add to version 3, and what would you need to learn to build it?

### 2.1.11 Optional Extension

Use the `time` module to slow the bot down. A short pause before a reply makes it feel like the bot is thinking instead of dumping text. You decide where the pauses belong and how long they run — too many is annoying, too few changes nothing.

## 3 Identify the Assessed Concepts

### 3.1 From Chapter 9: Collections

- **Lists and elements.** Four lists do two different jobs. Two hold possible replies the bot can choose from; two hold the answers a question will accept. Same structure, different purpose, and the reflection asks students to articulate the difference.
- **Iterating through a collection.** A `for` loop walks the menu list and prints each option. This is the smallest honest use of iteration I could find: the loop is three lines, it replaces three `print` statements, and adding a fourth option costs nothing.
- **Membership testing.** `while home not in HOMES` asks a question about the whole list rather than about one element. This is the operation that makes a list worth having here — with three separate

variables there is nothing to ask.

- **Random selection from a collection.** `random.choice()` treats a list as a bag to draw from. Students see a third thing a list can be for, and it is the one that makes the bot feel alive.

### 3.2 From Chapter 10: Functions and Decomposition

- **Functions with parameters.** The asking function takes the question text as a parameter, which is what lets one function serve nine questions.
- **Return values.** The function returns rather than prints, so the answer can be stored, compared against a list, and used in an f-string later. This is the idea the whole assignment turns on.
- **Decomposition.** The measure of success is in the criteria: adding a ninth question should cost one line. That is the test of whether the decomposition is real, or whether the student wrapped a single call in a `def` and moved on.

### 3.3 Retained from earlier chapters

The `while` loop is prior knowledge, not new material, but it is doing new work. Students have used `while` to count. Here it guards a question, and the condition is a membership test against a list — which is how Chapter 9 and the earlier loops chapter end up in the same four lines of code.

## 4 Provide a Detailed Specification

### 4.1 Program Behavior

1. Ask for the user's name through the asking function, then capitalize it for display.
2. Greet the user by name.
3. For each of two menu questions: print the question, loop over its option list to display the choices, ask, and keep asking while the answer is not in the list.
4. For each of six free-text questions: ask and store the answer.
5. Run at least four decision blocks — two on the menu answers, two on free-text answers. At least two use `elif`; all end with `else`.
6. Between questions, print a randomly chosen line from one of the two reply lists.
7. Print a closing summary that reports back what the person said, using f-strings throughout.

### 4.2 Input

All input is typed by the user at `input()` prompts, always through the asking function: one name, then eight answers. Two of those eight are constrained to a fixed list of three options and will be requested repeat-

edly until they match. The other six are free text, which is why every decision block on them needs an `else`.

### 4.3 Processing

Every answer is stripped of surrounding whitespace and lowercased inside the function, before it is returned. This matters more than it did in version 1: a menu answer is now compared against a list with `in`, and "CALM " fails that test while "calm" passes. Cleaning is no longer a nicety that makes the decision blocks tidier — it is what keeps a correctly-answering user out of an unnecessary retry.

The name is the one answer given further treatment at the call site: `ask(...).title()` takes the cleaned lowercase string and capitalizes it, so it prints the way a name should regardless of how it was typed.

### 4.4 Output

Printed text only:

- a greeting using the name;
- a printed menu of options before each constrained question;
- a refusal message naming what the person typed, whenever it is not on the list;
- one response per decision block;
- filler lines chosen at random between questions;
- a closing summary quoting the person's answers back.

### 4.5 Collection Specification

Four lists, two roles.

| Name          | Stores                                  | Size | How accessed                                                |
|---------------|-----------------------------------------|------|-------------------------------------------------------------|
| THINKING      | Short acknowledgement lines             | 3    | <code>random.choice()</code>                                |
| ENCOURAGEMENT | Short reassurance lines                 | 3    | <code>random.choice()</code>                                |
| HOMES         | Accepted answers to "where do you live" | 3    | <code>for</code> loop to print; <code>in</code> to validate |
| ENERGIES      | Accepted answers to "what energy"       | 3    | <code>for</code> loop to print; <code>in</code> to validate |

All four are fixed at the top of the program and never modified while it runs. None of them needs to be, which is worth saying plainly: the assignment does not require a list that grows, because nothing in this program has a good reason to grow one.

## 4.6 Function Specification

| Field     | ask                                               |
|-----------|---------------------------------------------------|
| Purpose   | Ask one question and hand back a usable answer    |
| Parameter | <code>question</code> — the text to show the user |
| Returns   | The user's answer, stripped and lowercased        |

One function is required, and this is it. Students are required to write a function matching `ask` in job, though not in name.

## 4.7 Algorithm

```
BEGIN
  DEFINE ask(question)
    answer <- input(question)
    RETURN answer stripped and lowercased

  SET THINKING and ENCOURAGEMENT to lists of possible replies
  SET HOMES and ENERGIES to lists of accepted answers

  name <- ask(greeting question), capitalized
  DISPLAY welcome using name

  FOR each menu question (HOMES, then ENERGIES):
    DISPLAY the question
    FOR each option IN the option list
      DISPLAY the option
      answer <- ask(prompt)
      WHILE answer NOT IN the option list
        DISPLAY refusal naming what was typed
        answer <- ask(prompt)
      IF / ELIF / ELSE on the answer
        DISPLAY the matching response

  FOR each free-text question:
    answer <- ask(question)
    OCCASIONALLY display random.choice(THINKING or ENCOURAGEMENT)
    IF the question has a decision block
      IF / ELIF / ELSE on the answer
        DISPLAY the matching response

  DISPLAY closing summary using the stored answers
END
```

## 5 Include Example Input and Output

Four cases. The first is typical; the second and third are the special and boundary cases; the fourth is the one that reveals a common error.

## 5.1 Case 1 — Typical

Valid menu answers on the first try, ordinary text everywhere else. Every decision block lands on a branch that fits the answer.

Welcome to the shelter! What is your name? Russell  
Good to meet you, Russell. Let me find you a friend.

Where do you live?

- house
- apartment
- dorm

> apartment

An apartment narrows things down, but it does not rule much out.

How much room do you have for a pet? not much  
Noted.

Are you home most of the day, or out most of the day? out most of the day  
Then we want somebody who is comfortable on their own.

What kind of energy are you hoping for?

- calm
- playful
- somewhere in between

> calm

Calm it is. I have three in mind already.

How do you feel about early mornings? I hate them  
Let me write that down.

Have you had a pet before? yes  
Experience helps. You know what you are signing up for.  
Every animal in here is hoping you walk in.

What is one thing you are looking forward to? coming home to somebody  
Anything you are nervous about? the vet bills honestly  
Okay, that helps.

Here is what I have, Russell.

Apartment, not much to work with, and you are out most of the day.

You want calm. On early mornings you said: i hate them.

Looking forward to coming home to somebody. Nervous about the vet bills honestly.

Give me a minute. I think I know who to bring out.

The filler lines vary between runs. That is the `random.choice()` requirement working, and it is worth pointing out to students that a correct program does not have to produce identical output twice.

## 5.2 Case 2 — Special: messy typing

Capitals and stray spaces on every answer. The cleaning inside `ask` has to absorb them, or the menu answers get refused and every decision block falls through to its `else`.

```
Answers typed: russell / "  APARTMENT  " / Some / "  Home  " /  
               "  CALM  " / fine / "  YES  " / walks / nothing
```

```
Good to meet you, Russell. Let me find you a friend.  
An apartment narrows things down, but it does not rule much out.  
Being around a lot is the single best thing you can offer.  
Calm it is. I have three in mind already.  
Experience helps. You know what you are signing up for.
```

```
Apartment, some to work with, and you are home.
```

Every block matches despite the mess, and no retry is triggered. The name still prints capitalized, because `.title()` is applied to the cleaned string at the call site rather than inside the function. This case is the argument for why cleaning belongs in the function: it happens once, and every comparison downstream inherits it.

## 5.3 Case 3 — Boundary: the answer is not on the menu

Two wrong answers on the first menu and one on the second. The `while` loop has to refuse each one and ask again without moving on.

```
Where do you live?  
  - house  
  - apartment  
  - dorm  
> condo  
I do not have "condo" on the list. Pick one of the three.  
> tent  
I do not have "tent" on the list. Pick one of the three.  
> apartment  
An apartment narrows things down, but it does not rule much out.  
  
...
```

```
What kind of energy are you hoping for?  
  - calm  
  - playful  
  - somewhere in between  
> chill  
I do not have "chill" on the list. Pick one of the three.  
> playful  
Playful is easy to find here. Loud, but easy.
```

This is the case that separates a working retry loop from a decorative one. A student who prints the refusal but forgets to ask again inside the loop gets the message printed forever; a student who asks again but

outside the loop gets exactly one extra try. Neither shows up in Case 1.

## 5.4 Case 4 — Error: the user answers nothing

Enter pressed on every question, including twice on each menu.

```
>
I do not have "" on the list. Pick one of the three.
> house
A house gives us plenty to work with.
```

```
How much room do you have for a pet?
Noted.
```

```
Are you home most of the day, or out most of the day?
I will assume your days are a mix.
```

```
...
```

```
Here is what I have, Ana.
House, to work with, and you are .
You want calm. On early mornings you said: .
Looking forward to . Nervous about .
```

Nothing crashes. The two menu questions refuse the blank and ask again, which is the retry requirement doing its job. The six free-text questions accept it, and the closing summary prints the gaps.

That split is the most interesting thing in the test set, and it is not a bug. Validation protects the questions I chose to validate, and I chose two. The cost of validating all eight would be a bot that argues with you about what you are nervous about. This is precisely what the fourth reflection prompt asks students to reason about, and now I can point at a transcript when I ask it.

## 6 Create the Solution

### 6.1 Source Code

---

```
"""Chatbot 2.0 by Mr. Helmstedter: the Shelter Matchmaker."""

import random

def ask(question):
    """Print a question and hand back the answer with the mess cleaned off."""
    answer = input(question)
    return answer.strip().lower()

THINKING = [
    "Let me write that down.",
    "Noted.",
```

```

    "Okay, that helps.",
]

ENCOURAGEMENT = [
    "You are going to be a great match for somebody.",
    "Every animal in here is hoping you walk in.",
    "No wrong answers on this one.",
]

HOMES = ["house", "apartment", "dorm"]
ENERGIES = ["calm", "playful", "somewhere in between"]

name = ask("Welcome to the shelter! What is your name? ").title()
print(f"Good to meet you, {name}. Let me find you a friend.")
print()

print("Where do you live?")
for option in HOMES:
    print(f"    - {option}")
home = ask("> ")
while home not in HOMES:
    print(f'I do not have "{home}" on the list. Pick one of the three.')
    home = ask("> ")

if home == "house":
    print("A house gives us plenty to work with.")
elif home == "apartment":
    print("An apartment narrows things down, but it does not rule much out.")
else:
    print("A dorm is tight, but people make it work.")
print()

space = ask("How much room do you have for a pet? ")
print(random.choice(THINKING))
print()

schedule = ask("Are you home most of the day, or out most of the day? ")
if "home" in schedule:
    print("Being around a lot is the single best thing you can offer.")
elif "out" in schedule:
    print("Then we want somebody who is comfortable on their own.")
else:
    print("I will assume your days are a mix.")
print()

print("What kind of energy are you hoping for?")
for option in ENERGIES:
    print(f"    - {option}")
energy = ask("> ")
while energy not in ENERGIES:
    print(f'I do not have "{energy}" on the list. Pick one of the three.')
    energy = ask("> ")

if energy == "calm":
    print("Calm it is. I have three in mind already.")
elif energy == "playful":
    print("Playful is easy to find here. Loud, but easy.")

```

```

else:
    print("Somewhere in the middle. That is most of them, honestly.")
print()

mornings = ask("How do you feel about early mornings? ")
print(random.choice(THINKING))
print()

experience = ask("Have you had a pet before? ")
if experience in ["yes", "y", "yeah", "yep"]:
    print("Experience helps. You know what you are signing up for.")
elif experience in ["no", "n", "nope"]:
    print("First time is exciting. We will start you somewhere gentle.")
else:
    print("Either way, we will figure it out together.")
print(random.choice(ENCOURAGEMENT))
print()

hoping_for = ask("What is one thing you are looking forward to? ")
nervous = ask("Anything you are nervous about? ")
print(random.choice(THINKING))
print()

print(f"Here is what I have, {name}.")
print(f"{home.title()}, {space} to work with, and you are {schedule}.")
print(f"You want {energy}. On early mornings you said: {mornings}.")
print(f"Looking forward to {hoping_for}. Nervous about {nervous}.")
print()
print("Give me a minute. I think I know who to bring out.")

```

---

## 6.2 Evidence of Testing

Rather than retype output by hand, I wrote a harness that feeds each case to the real program through standard input and checks every decision-block response and closing line against what the specification promises. It fixes `random.seed` so the filler lines are reproducible. The messy-typing case also asserts that a refusal message does *not* appear, since a passing run there is one where no retry was needed. The file is `test_chatbot_2.py`. Its actual output follows.

[PASS] Typical case

Valid menu answers on the first try, ordinary text everywhere else.  
Every decision block should land on a branch that fits the answer.

[PASS] Special case: messy typing

Capitals and stray spaces on every answer. The cleaning inside `ask()` has to absorb them, or the menu answers get refused and every decision block falls through to its else.

[PASS] Boundary case: answer not on the menu

Two wrong answers on the first menu and one on the second. The while loop has to refuse each one and ask again without moving on.

[PASS] Error case: user answers nothing

Enter pressed on every free-text question, and once on each menu

```
before a real answer. The program does not crash, but the closing
summary prints gaps where the answers should be.
```

4 of 4 test cases passed.

The harness is teacher infrastructure, not a student requirement. Students test by talking to their bot.

## 7 Create Teacher Notes

### 7.1 Required Prior Knowledge

Variables, `input()`, f-strings, string methods, and `if~elif~else` from the first chatbot. Lists, `for` loops, `while` loops, functions, and the `random` module from the notes notebooks. Students who have not written a function before will not be able to start this.

### 7.2 New Vocabulary

Parameter, argument, return value, call, element, membership, iterate. *Parameter* and *argument* are the pair students conflate; the distinction is worth ten seconds and a diagram, not a lecture.

### 7.3 Suggested Scaffolding

Open by projecting a version 1 with the eight ask-and-clean stretches visible at once, scrolled slowly. The repetition is the argument for the whole assignment and it lands better seen than described.

Requirements 3 and 4 are best introduced together, at the board, in that order. Print the menu with a `for` loop first and let students try it. Somebody will type something that is not on the menu within thirty seconds, and the bot will sail past it. That is the moment to introduce the `while` loop — not as a new construct, but as the fix for a problem the room just watched happen.

### 7.4 A Likely Loop Misconception

The infinite retry. A student writes the refusal message inside the `while` loop and the second `ask` outside it, or forgets the second `ask` entirely. The condition never changes, so the message prints forever and Colab has to be interrupted.

This is the most likely crash-equivalent in the assignment, and it is worth pre-empting by naming the rule out loud: whatever the `while` is checking, something inside the loop has to change it. Ask them to point at the line that changes it before they run the cell.

### 7.5 A Likely Function Misconception

Students print inside the function instead of returning from it. It looks like it works — the right words appear on screen — so nothing prompts them to fix it. Then they try to use the result and find they have nothing to use, because the function handed back `None`.

The fastest way to show this is to run `x = ask("Hi? ")` and then `print(x)` with a printing version: the answer appears twice, then `None`. It is more urgent this time than it was in version 1, because the returned value now feeds a membership test — `while None not in HOMES` is an infinite loop, so the printing mistake and the looping mistake collide.

## 7.6 A Note on the else Branches

Both menu questions have exactly three options, so `if / elif / else` covers them with nothing left over. The `else` is the third valid option, not an error catch — the `while` loop already ate the bad input. This is worth mentioning when reviewing exemplars, because students who learned `else` as “everything that went wrong” will read it incorrectly here.

## 7.7 Accessibility

Colab runs in any browser, works with a screen reader, and inherits browser font sizing. Because it is cloud-hosted, this must be an in-class assignment for any student without home internet, and the three-period estimate assumes that.

Students who find the free-text conversation overwhelming to invent can be given a question list to start from; the programming requirements are unchanged by whose questions they are.

## 7.8 Support for Students Who Need More Guidance

Give them their own version 1 and a single instruction: make the asking part happen once. That is a smaller ask than a rebuild and it produces the same insight. The menus and the retry loops can follow in the second and third periods.

A pre-written `ask` function is available for students who need it, but I would hand it over late rather than early — the struggle to extract it is the thing being learned.

## 7.9 Extension for Students Ready for More

The `time` extension in the assignment is the first step. Past that: let the bot accept short forms by checking the answer against a second list of synonyms before refusing it, or count how many tries the person needed and mention it at the end.

The obvious extension I am deliberately not requiring is folding the menu-and-retry block into its own function. A student who notices they wrote the same six lines twice and asks about it has found the next thing to learn, and I would rather answer that question than assign it.

## 8 Reflection

### 8.1 Why is the assignment appropriate for this grade level?

This is the second draft of something they already built. The hardest part of teaching functions is that the motivating problem — repeated code — usually has to be invented by the teacher and handed to students, who then take it on faith. Here they wrote the repetition themselves, weeks ago, and it is still sitting in their Drive. When I ask what would happen if they added two more questions, they know, because they remember what adding the first eight felt like.

The second half of the assignment is appropriate for a different reason. Requirements 3 and 4 are not about code at all; they are about the person on the other side of the conversation. A high school student has used enough software to have opinions about which forms are annoying, and this is the first assignment where those opinions are actionable. Asking a fifteen-year-old to decide which two questions deserve a second chance is asking them to make a design judgment, and they are more than ready for it.

### 8.2 How does the platform support or limit the Chapter 9 and 10 concepts?

Colab supports them fully. Python lists and functions are the concepts, not approximations of them, and there is nothing in Chapter 9 or 10 that Python makes awkward.

Where Colab limits things is in what students can see. A notebook does not show you a list; it shows you what you printed. The mitigation is teaching `print(HOMES)` as a debugging habit early, which is worth doing anyway.

The cell model is a real hazard for this assignment in a way it was not for the last one. A student who defines a function, edits it, and forgets to re-run the cell is calling the old version, which produces bugs that survive a correct fix. Worse, the retry loops mean a student can leave a cell running and waiting for input without realizing it — Colab shows a spinner, the student assumes the notebook is broken, and restarts. I plan to demonstrate an interrupt on day one.

### 8.3 How does the assignment help students organize data or decompose a problem?

The four lists carry the organization argument, and they carry it better than a single list of answers would. I tried that first: collect every answer into one list as the conversation runs. It did not survive a simple question: why is a list better than eight variables here? The honest answer was that it was not. Every answer was already in a variable, the program pulled each one back out by index, and the only whole-list operation I could invent was contrived.

What replaced it is four small lists doing two genuinely different jobs. **THINKING** and **ENCOURAGEMENT** are bags to draw from. **HOMES** and **ENERGIES** are sets of acceptable answers, printed with a loop and then tested against with `in`. Neither job is possible with separate variables, and both are three lines long. The reflection prompt asks students to say what each list is holding and why a list is the right place for it, and the answer differs between the two kinds, which is the point.

The function carries the decomposition argument. The success criterion I care most about is not in the code at all — it is “adding a ninth question would take you one line.” That is the test of whether the

decomposition is real or whether the student just wrapped one call in a `def`.

## 8.4 What errors did I discover while testing?

Three, and the smallest one was the most embarrassing.

The closing summary originally read `f"A {home}, ..."`, which prints “A apartment.” I only caught it because I ran the thing. It is the kind of bug that no test case would flag, because nothing is wrong except the English, and the fix — drop the article and capitalize the word — took ten seconds. I mention it because my students produce this bug constantly and I have been describing it to them as carelessness. It is not. It is what happens when you write a sentence whose pieces you do not control.

The test harness itself had a real bug, and it taught me something about the assignment. My boundary case fed answers in the wrong order, because a retry consumes an input without advancing to the next question. A program with retry loops does not have a fixed-length input sequence, so any script that feeds it has to account for the rejections. I had written the case as though the bad answers were free. The harness caught it as a failure, and the fix was to count the retries.

The empty-answer case produced the finding I will actually use with students. Blank input is refused on the two menu questions and accepted on the other six, so the closing summary prints with gaps in it. Nothing crashed. The code is behaving exactly as specified. But the output is plainly broken to anybody reading it, and the reason is that I validated two questions and not the other six. That gap between “runs without errors” and “works” is not something I can lecture into students, and here it is, reachable in fifteen seconds, in a transcript I can project.

## 8.5 What would I revise before using it?

The duplicated menu block bothers me and I left it in anyway. Printing a menu and guarding it with a `while` loop takes six lines, and the solution contains those six lines twice, which sits badly in a project whose whole argument is that repetition is a problem to be removed. The fix is a second function that takes the question and the option list. I decided against requiring it — a function containing a loop, called later, is a step too far for students three weeks into functions, and requiring it would turn requirement 3 into the hardest thing in the assignment rather than the most approachable. So the repetition stays, visibly, and I will let it be the answer to the version 3 prompt. I am not certain this is right. It may read to a careful student as the teacher not following his own advice.

The other thing I would watch is the requirement count. Four new requirements plus a version 1 floor is a lot to hold, and moving version 1 off the rubric was my attempt to fix that — it is now a gate rather than a scored criterion, on the reasoning that a bot which does not clear it should come back for revision rather than receive a number. That decision makes the rubric shorter and the stakes clearer, but it also means a student who half-finishes gets no partial credit for the half that works. I think that is correct for this assignment and I would not extend it to a harder one.

## 9 Files

- `chatbot_2.py` — the complete solution, tangled from this document
- `test_chatbot_2.py` — the harness that produced the evidence above
- `Chatbot_2.ipynb` — the student-facing assignment notebook
- `Chatbot 2.0 Rubric` — the Google Sheets rubric, imported into Google Assignments